

R3.01 - DÉVELOPPEMENT WEB

Sujet TP 03 : les requêtes HTTP

🏠 BUT Informatique, 2^e année ⌚ 3 séances 👤 Desbenoit B.

L'objectif de ce TP est d'utiliser des requêtes HTTP pour afficher de manière dynamique les Vélib disponibles dans la ville de Paris pour chacune des stations.

Objectif

L'objectif du TP est de mettre en œuvre l'utilisation de requêtes HTTP pour modifier dynamiquement le contenu d'une page. Ici, nous allons nous concentrer exclusivement sur les éléments prévus par la norme du W3C. Les requêtes XHR seront faites en respectant la spécification CORS (Cross-Origin Resource Sharing) qui permet l'échange de données entre domaines.

Prérequis

Avant de commencer le TP il vous faut a minima webpack avec la configuration mise en place lors des précédents TP. Si vous n'y êtes pas parvenu, vous trouverez ci-dessous ma configuration. Elle est perfectible, mais au moins elle vous permettra de commencer rapidement.

Cette configuration correspond à un projet qui utilise des variables d'environnement (<https://www.npmjs.com/package/dotenv>) et sass (<https://sass-lang.com/>), et dont la structure est la suivante :

```
MonProjet
├── package.json
├── webpack.common.js
├── webpack.dev.js
├── webpack.prod.js
├── .env
├── src
│   ├── index.js
│   ├── images
│   │   ├── favicon.ico
│   │   └── ...
│   ├── scripts
│   │   ├── app.js
│   │   └── tpl
│   │       ├── main.tpl.html
│   │       └── popupview.tpl.html
│   └── stylesheets
│       └── styles.scss
```

Attention, il faut penser dans ce cas à faire l'import de la feuille de style dans le fichier `index.js` ou alors, l'ajouter comme point d'entrée dans la configuration de webpack.

webpack.common.js

```
import path from 'node:path'
import HtmlWebpackPlugin from 'html-webpack-plugin'
import Dotenv from 'dotenv-webpack'

export default {
  entry: './src/index.js',
  output: {
    path: path.resolve(import.meta.dirname, 'dist'),
    clean: true
  },
  plugins: [
    new HtmlWebpackPlugin({
      title: 'TP03',
      favicon: path.resolve(import.meta.dirname, './src/images/favicon.ico')
    }),
    new Dotenv()
  ],
  module: {
    rules: [
      {
        test: /\.js$/i,
        exclude: /node_modules/,
        use: {
          loader: 'babel-loader',
          options: {
            presets: ['@babel/preset-env']
          }
        }
      },
      {
        test: /\.s[ac]ss$/i,
        type: "css/auto",
        use: [
          {
            loader: "sass-loader",
            options: {
              sassOptions: {
                quietDeps: true
              }
            }
          }
        ]
      },
      {
        test: /\.tpl\.html$/i,
        type: 'asset/source'
      },
      {
        test: /\. (png|jpe?g|gif|svg|webp)$/i,
        type: 'asset/resource',
        generator: {
          filename: 'images/[name].[contenthash][ext]'
        }
      },
      {
        test: /\. (woff2?|ttf|otf|eot)$/i,
        type: 'asset/resource',
        generator: {
          filename: 'fonts/[name].[contenthash][ext]'
        }
      }
    ]
  }
}
```

```
},  
experiments: {  
  css: true,  
}  
}
```

webpack.dev.js

```
import { merge } from 'webpack-merge'  
import commun from './webpack.common.js'  
import 'dotenv/config';  
  
export default merge(commun, {  
  mode: 'development',  
  devtool: 'eval-source-map',  
  output: {  
    filename: 'js/[name].js',  
    cssFilename: 'css/[name].css'  
  },  
  devServer: {  
    port: 8080,  
    open: true,  
    hot: true,  
    watchFiles: ['src/**/*.html'],  
    proxy: [  
      {  
        context: ['/api'],  
        target: process.env.API_URL,  
        changeOrigin: true,  
        pathRewrite: { '^/api': '' }  
      }  
    ]  
  }  
})
```

webpack.prod.js

```
import { merge } from 'webpack-merge'  
import commun from './webpack.common.js'  
  
export default merge(commun, {  
  mode: 'production',  
  devtool: 'source-map',  
  output: {  
    filename: 'js/[name].[contenthash].js',  
    cssFilename: 'css/[name].[contenthash].css'  
  }  
})
```

Et mon **package.json** (à adapter à vos besoins) :

```
{
  "name": "correction_tp03_2026",
  "version": "1.0.0",
  "description": "",
  "private": true,
  "type": "module",
  "scripts": {
    "start": "webpack serve --config webpack.dev.js",
    "build": "webpack --config webpack.prod.js"
  },
  "keywords": [],
  "author": "Desbenoit B.",
  "license": "ISC",
  "devDependencies": {
    "@babel/core": "^8.0.6",
    "@babel/preset-env": "^8.0.6",
    "babel-loader": "^10.1.1",
    "dotenv": "^18.0.3",
    "dotenv-webpack": "^9.0.0",
    "handlebars": "^4.7.9",
    "html-webpack-plugin": "^5.6.8",
    "leaflet": "^1.9.4",
    "leaflet.markercluster": "^1.5.3",
    "material-icons": "^1.13.14",
    "materialize-css": "^1.0.0",
    "sass": "^1.105.0",
    "sass-loader": "^17.0.1",
    "webpack": "^5.111.1",
    "webpack-cli": "^7.2.3",
    "webpack-dev-server": "^6.0.0",
    "webpack-merge": "^6.0.1"
  },
  "dependencies": {
    "npm-check-updates": "^23.1.0"
  },
  "browserslist": [
    "> 0.5%",
    "last 2 versions",
    "not dead"
  ]
}
```

Et dans `.env`

```
API_URL = 'https://velib-metropole-opendata.smovengo.cloud'
```

Ces deux derniers éléments (le fichier `.env` et le `proxy` dans `webpack.dev.js`) sont nouveaux par rapport aux TP précédents. Ils sont expliqués en détail dans la section suivante : prenez le temps de la lire avant de commencer à coder, cela vous évitera de longues minutes de débogage.

Comprendre le fichier `.env` et le proxy

Le fichier `.env` : les variables d'environnement

À quoi ça sert ?

Une application dépend souvent de valeurs qui **changent selon l'endroit où elle s'exécute** : l'URL d'une API (serveur de test ou de production), une clé d'accès, un numéro de port, etc. Écrire

ces valeurs « en dur » dans le code pose plusieurs problèmes :

- pour changer de serveur, il faut modifier le code source (et risquer d'en oublier un endroit) ;
- chaque développeur de l'équipe peut avoir besoin de valeurs différentes sur sa machine ;
- les informations sensibles (mots de passe, clés d'API) finissent dans le dépôt Git, visibles par tous.

La solution classique consiste à placer ces valeurs dans des **variables d'environnement**, et à les regrouper dans un fichier texte nommé `.env`, situé **à la racine du projet** (au même niveau que `package.json`). Chaque ligne associe un nom à une valeur :

```
NOM_DE_LA_VARIABLE=valeur
```

Par convention, les noms sont écrits en majuscules avec des `_` comme séparateurs. Ici, nous n'avons besoin que d'une seule variable, `API_URL`, qui contient l'adresse du serveur de l'API.

Comment ces variables sont-elles lues ?

Dans notre configuration, le fichier `.env` est lu par **deux outils différents**, pour deux usages différents :

1. **dotenv**, importé dans `webpack.dev.js` avec la ligne `import 'dotenv/config'`. Au démarrage, ce paquet lit le fichier `.env` et place chaque variable dans l'objet `process.env` de Node.js. C'est ce qui permet d'écrire `target: process.env.API_URL` dans la configuration du proxy. Ce code s'exécute **dans Node.js, sur votre machine**, au moment où webpack démarre.
2. **dotenv-webpack**, ajouté comme plugin dans `webpack.common.js` (`new Dotenv()`). Ce plugin sert à rendre les variables utilisables **dans votre code front-end** (le code de `src/`). Au moment de la compilation, webpack remplace chaque occurrence de `process.env.NOM_DE_LA_VARIABLE` par sa valeur. Par exemple, si vous écrivez dans `app.js` :

```
console.log(process.env.API_URL)
```

le fichier JavaScript produit par webpack contiendra en réalité :

```
console.log('https://velib-metropole-opendata.smovengo.cloud')
```

Le navigateur, lui, ne connaît ni `process` ni le fichier `.env` : c'est webpack qui fait le remplacement avant que le code n'arrive dans le navigateur. Seules les variables effectivement utilisées dans le code sont ainsi injectées.

Points de vigilance

- **Redémarrez le serveur** (`npm start`) après chaque modification du fichier `.env` : il n'est lu qu'au lancement de webpack.
- **Ne versionnez pas le fichier `.env`** : ajoutez-le à votre fichier `.gitignore`. Pour que vos camarades sachent quelles variables définir, l'usage est de versionner à la place un fichier `.env.example` qui contient les noms des variables avec des valeurs d'exemple (ou vides).
- **Tout ce qui est injecté dans le code front-end est public**. Une variable utilisée dans `src/` se retrouve en clair dans le JavaScript téléchargé par le navigateur : n'importe qui peut la lire avec

les outils de développement. Le fichier `.env` permet de ne pas stocker une clé secrète dans Git, mais il ne la rend pas secrète une fois qu'elle est envoyée au navigateur.

Le proxy : contourner le blocage CORS

Le problème : la politique de même origine

Pour des raisons de sécurité, les navigateurs appliquent la **politique de même origine** (*Same-Origin Policy*). L'**origine** d'une page est définie par le triplet **protocole + nom de domaine + port**. Pendant le TP, votre application est servie par webpack à l'adresse `http://localhost:8080`, alors que l'API se trouve sur `https://velib-metropole-opendata.smovengo.cloud` : les origines sont différentes (protocole, domaine et port changent).

Lorsque votre code JavaScript fait une requête vers une autre origine (avec `fetch` ou `XMLHttpRequest`), le navigateur envoie bien la requête, mais il **n'autorise votre code à lire la réponse que si le serveur l'accepte explicitement**. Pour cela, le serveur doit ajouter dans sa réponse l'en-tête HTTP `Access-Control-Allow-Origin` : c'est le mécanisme **CORS** (*Cross-Origin Resource Sharing*). Si cet en-tête est absent, la requête échoue et vous obtenez dans la console un message de ce type :

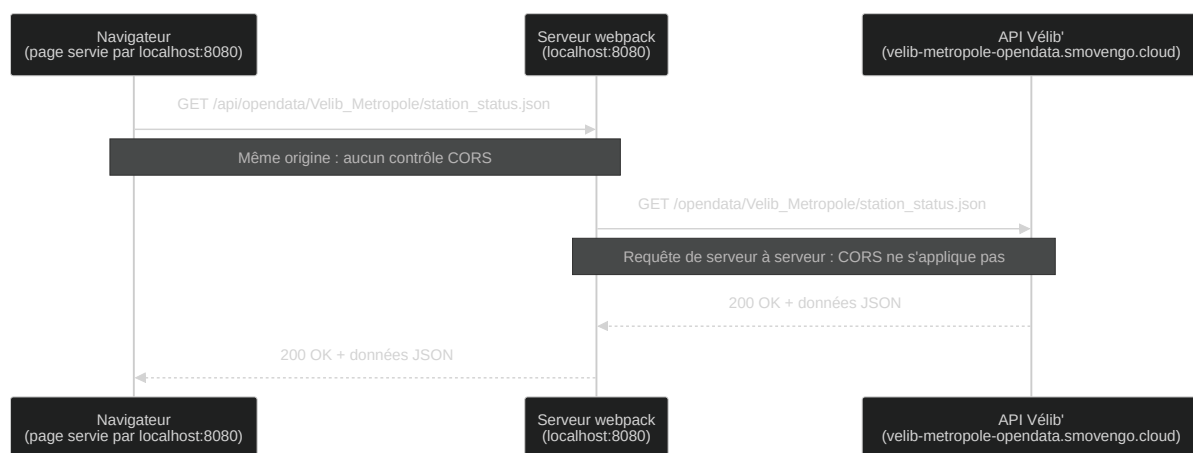
```
Access to fetch at 'https://velib-metropole-opendata.smovengo.cloud/...' from origin 'http://localhost:8080' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

Vous ne pouvez rien y faire dans votre code JavaScript : c'est au serveur de l'API de décider s'il autorise les autres origines, et vous n'en avez pas le contrôle.

Point important : **CORS est une protection appliquée uniquement par le navigateur**. Si vous testez la même URL directement dans la barre d'adresse, avec `curl` ou avec Postman, tout fonctionne. C'est cette particularité que nous allons exploiter.

La solution : un intermédiaire côté serveur

L'idée du proxy est simple : puisque le navigateur refuse de lire les réponses d'une autre origine, on fait en sorte qu'il ne s'adresse **qu'à sa propre origine**, c'est-à-dire au serveur de développement de webpack. C'est ensuite ce serveur (un programme Node.js, qui n'est pas soumis aux règles CORS) qui va chercher les données auprès de l'API et les renvoie au navigateur.



Pour le navigateur, la requête et la réponse viennent de `localhost:8080` : il n'y a plus de re-

quête « cross-origin », donc plus de blocage.

Lecture de la configuration

Reprenons le bloc `proxy` de `webpack.dev.js` :

```
proxy: [
  {
    context: ['/api'],
    target: process.env.API_URL,
    changeOrigin: true,
    pathRewrite: { '^/api': '' }
  }
]
```

- `context: ['/api']` : seules les requêtes dont le chemin commence par `/api` sont interceptées et transmises par le proxy. Toutes les autres (votre page HTML, vos scripts, vos images...) sont servies normalement par webpack.
- `target: process.env.API_URL` : l'adresse du serveur vers lequel les requêtes sont redirigées, lue depuis le fichier `.env` (voir plus haut).
- `changeOrigin: true` : modifie l'en-tête `Host` de la requête transmise pour qu'il corresponde au serveur cible plutôt qu'à `localhost:8080`. Beaucoup de serveurs (en particulier en HTTPS) refusent les requêtes dont l'en-tête `Host` ne correspond pas à leur nom de domaine.
- `pathRewrite: { '^/api': '' }` : supprime le préfixe `/api` avant de transmettre la requête. Ce préfixe ne sert qu'à dire au serveur webpack « cette requête est pour le proxy » ; l'API, elle, ne le connaît pas.

Concrètement, voici ce que devient une requête :

Dans votre code	Requête réellement envoyée à l'API
<code>fetch('/api/opendata/Velib_Metropole/station_information.json')</code>	<code>https://velib-metropole-opendata.smovengo.cloud/opendata/Velib_Metropole/station_information.json</code>

Ce que cela change dans votre code

Dans votre code front-end, vous devez donc **toujours utiliser une URL relative commençant par `/api`**, et non l'URL complète de l'API :

```
// Passe par le proxy : même origine, pas de problème CORS
const reponse = await fetch('/api/opendata/Velib_Metropole/station_information.json')

// Requête directe vers une autre origine : bloquée par le navigateur
const reponse = await fetch('https://velib-metropole-opendata.smovengo.cloud/opendata/Velib_Metropole/station_information.json')
```

Pour vérifier que tout fonctionne, ouvrez l'onglet **Réseau** (*Network*) des outils de développement : vous devez voir des requêtes vers `http://localhost:8080/api/...` avec un code de statut `200`.

Attention : le proxy est une fonctionnalité du **serveur de développement** (`npm start`). Il n'existe pas dans la version construite avec `npm run build` , qui ne produit que des fichiers statiques dans le dossier `dist` . Pour mettre une telle application en production, il faudrait configurer un proxy équivalent sur le serveur web qui l'héberge (Apache, Nginx...), ou utiliser une API qui autorise explicitement les requêtes CORS.

Travail à réaliser

Vous avez 6 heures pour programmer une application monopage (<https://developer.mozilla.org/en-US/docs/Glossary/SPA>) qui permet d'afficher le nombre de vélos disponibles dans les stations Vélib', service de vélos partagés de la métropole du Grand Paris.

Pour cela, vous utiliserez l'API ouverte de Vélib' Métropole. Elle respecte le standard GBFS (<https://gbfs.org/>) (*General Bikeshare Feed Specification*), un format commun utilisé par de nombreux services de vélos partagés dans le monde. Le point d'entrée de l'API est le fichier `gbfs.json` , qui liste les différents flux de données disponibles :

`https://velib-metropole-opendata.smovengo.cloud/opendata/Velib_Metropole/gbfs.json`

Commencez par ouvrir cette URL directement dans votre navigateur pour explorer les données. Cela fonctionne sans problème de CORS, car ce n'est pas votre code JavaScript qui fait la requête. Deux flux vous seront utiles :

Flux	Contenu
<code>station_information.json</code>	les caractéristiques fixes de chaque station : identifiant (<code>station_id</code>), nom, position (<code>lat</code> , <code>lon</code>), capacité...
<code>station_status.json</code>	l'état en temps réel de chaque station : nombre de vélos disponibles (mécaniques et électriques), nombre de bornes libres...

Les deux flux ont en commun le champ `station_id` , qui permet d'associer les informations d'une station à son état.

N'oubliez pas que, depuis votre code, ces flux doivent être appelés **via le proxy** (voir la section précédente), par exemple : `fetch('/api/opendata/Velib_Metropole/station_status.json')` .

Dans un premier temps à partir d'une requête `http` , réalisée via l'objet `XMLHttpRequest` ou la fonction `fetch` , sur l'API citée précédemment, vous réaliserez un affichage simple des données dans la page web. Cet affichage pourra prendre la forme d'une liste `<ul>` . Vous pourrez ainsi, pour chaque station, créer un `<li>` en y ajoutant les informations de votre choix.

Une fois cela réalisé, je vous propose d'essayer d'afficher ces données directement sur une carte. Cela est possible, car les données sont géolocalisées (cf. les champs `lat` et `lon` du flux `station_information`). Pour cela, vous aurez besoin des éléments suivants :

1- une librairie JavaScript pour gérer une carte interactive : <http://leafletjs.com/> (<http://leafletjs.com/>) (affichage d'un fond de carte, ajout de marqueurs, gestion de popup, etc.).

2- un fond de carte à afficher dans la librairie précédente : vous pouvez utiliser les fonds de carte d'OpenStreetMap (cf. documentation (<https://leafletjs.com/>))

Ce qui est intéressant ici, c'est que ces données sont actualisées toutes les minutes. Modifiez votre

application afin que les données de la carte soient mise à jour toutes les 5 minutes (inutile de surcharger le serveur). Pour cela je vous invite à regarder les éléments JavaScript suivant : `setTimeout` et `Map`. Il ne va peut-être pas paraître tout de suite évident que vous en avez besoin, aussi prenez le temps de réfléchir à leur intérêt ici.

Même si cela n'est pas clair pour vous au premier abord, la bibliothèque `leaflet` est fournie sous la forme d'une bibliothèque au format ES2015. Il est donc possible de l'importer. Pour cela, vous devez d'abord si ce n'est pas déjà fait l'ajouter dans vos dépendances :

```
npm install -D leaflet
```

Ensuite, il n'y a plus qu'à l'importer dans votre fichier js.

```
import * as L from 'leaflet'
```

Attention

Avec webpack, les versions 1.x de Leaflet affichent mal les marqueurs par défaut : l'icône n'apparaît pas et la console signale une erreur 404.

Leaflet déduit l'emplacement de ses images à partir d'un chemin présent dans sa feuille de style. Or webpack renomme et déplace ces images au moment du build, ce qui fausse ce calcul. La solution consiste à importer vous-même les images et à les fournir directement à Leaflet.

Pour corriger le problème, ajoutez le code suivant juste après l'import de la librairie (voir l'issue 4968 (<https://github.com/Leaflet/Leaflet/issues/4968>)) :

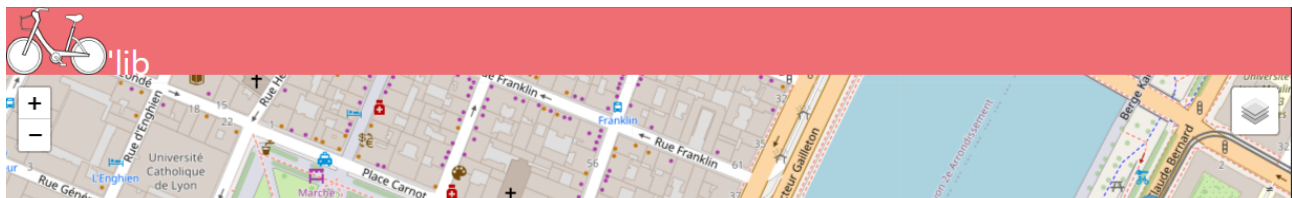
```
import iconUrl from 'leaflet/dist/images/marker-icon.png'
import iconRetinaUrl from 'leaflet/dist/images/marker-icon-2x.png'
import shadowUrl from 'leaflet/dist/images/marker-shadow.png'
delete L.Icon.Default.prototype._getIconUrl
L.Icon.Default.mergeOptions({ iconUrl, iconRetinaUrl, shadowUrl })
```

Enfin, pour que votre carte s'affiche correctement, vous aurez besoin de la feuille de style associée. Pour ça, il vous faudra ajouter dans votre fichier de style (css ou scss) l'import suivant :

```
@import url("leaflet/dist/leaflet.css");
```

Résultat attendu

Après un peu de travail, vous devriez pouvoir arriver au résultat ci-dessous. si vous n'arrivez pas à finir en séance, je vous invite fortement à terminer chez vous avant la prochaine séance de TP.





Pour aller un peu plus loin je vous propose de regarder du côté de standard JS (<https://standardjs.com/>) qui permet de définir des règles pour *ESLint* dont l'objectif est d'améliorer la qualité du code en sensibilisant les développeurs aux bonnes pratiques. Même si les règles sont discutables puisqu'il ne s'agit pas d'une norme, exercice reste très formateur.